



Longest Uniform Substring After Replacements

A **uniform** substring is one in which all characters are identical. Given a string, determine the **length** of the longest uniform substring that can be formed by **replacing up to k** characters.

Example:

a a ~~b~~^c c ~~d~~^c c c a $\longrightarrow$ a a c c c c c a
longest uniform substring

Input: `s = 'aabcdcca', k = 2`

Output: 5

Explanation: if we can only replace 2 characters, the longest uniform substring we can achieve is "ccccc", obtained by replacing 'b' and 'd' with 'c'.

Intuition

Determining if a substring is uniform

Before we try finding the longest uniform substring, let's first determine the most efficient way to make a string uniform with the fewest character replacements. Consider the example below:

a b a a b a c

Which characters should we replace to ensure the minimum number of replacements are performed to make the string uniform? There are three main choices: make the string all 'a's, or all 'b's, or all 'c's. The most efficient choice, requiring the fewest replacements, is to make all characters 'a', which involves just three replacements:

a ~~b~~^a a a ~~b~~^a a ~~c~~^a $\longrightarrow$ a a a a a a a

The key observation is that the minimum number of replacements needed to achieve uniformity is obtained by **replacing all characters except the most frequent one**.

This suggests that if we know the highest frequency of a character in a substring, we can determine if our value of `k` is sufficient to make that substring uniform. The number of characters that need to be replaced (`num_chars_to_replace`) can be found by subtracting this highest frequency from the total number of characters in the substring:

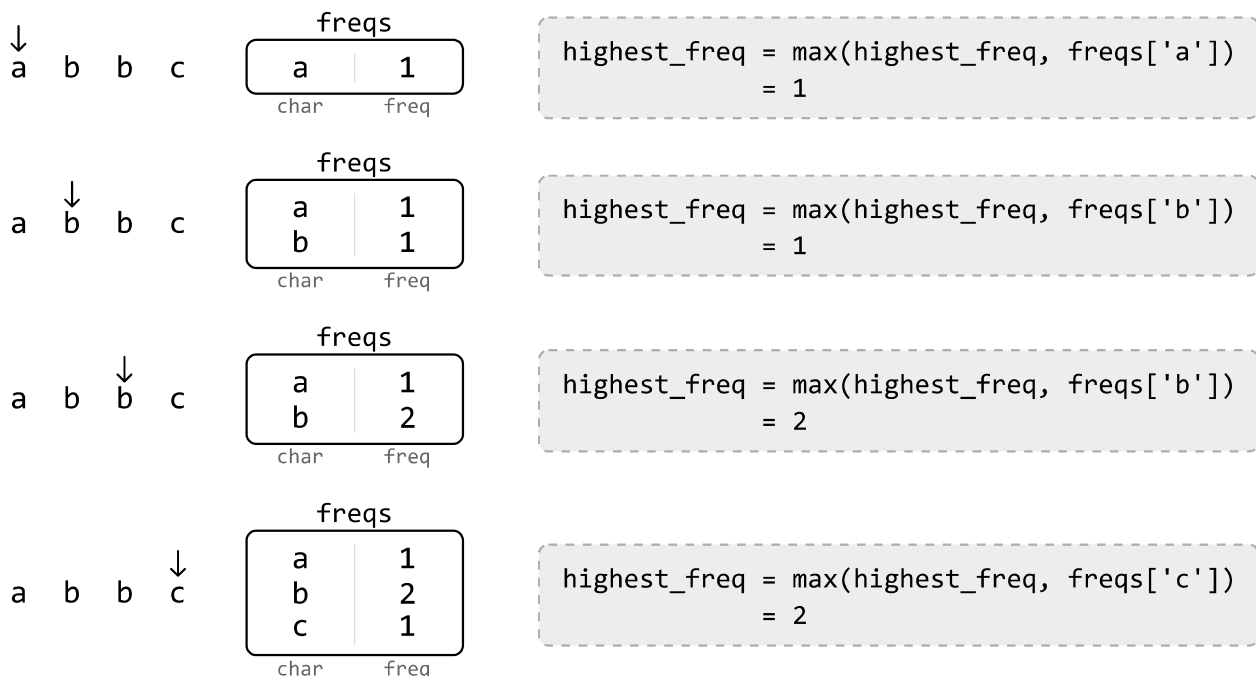
$$= 7 - 4$$

$$= 3$$

Once we've calculated `num_chars_to_replace` for a given substring, we can assess if the substring can be made uniform:

- If `num_chars_to_replace` $\leq k$, the substring can be made uniform.
- If `num_chars_to_replace` $> k$, the substring cannot be made uniform.

To calculate `num_chars_to_replace`, we need to know the value of `highest_freq`. This requires tracking the frequency of each character, which can be efficiently managed using a **hash map** (`freqs`). This hash map allows us to update `highest_freq` whenever we encounter a character with a higher frequency. Below is an illustration of how `freqs` is updated:



Now that we have the tools to determine if a substring can be made uniform, the next step is to figure out how to identify the longest uniform substring. Let's explore a technique that lets us do this.

Dynamic sliding window

We know sliding windows can be useful for solving problems involving substrings. This problem requires that we find the longest substring that satisfies a specific condition:

$$\text{num_chars_to_replace} \leq k$$

So, a dynamic sliding window might be appropriate, as discussed in the chapter introduction.

We can use the above condition to determine how to expand or shrink the window:

- If the condition is violated (i.e., the window is invalid), we **shrink** the window until it meets the condition again.

Let's see how this works over the example below:

a a b c d c c a, k = 2

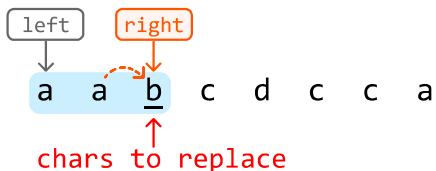
k = 2



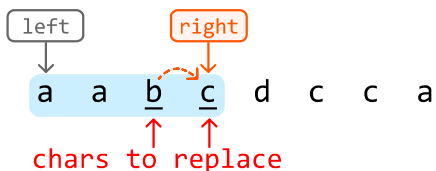
```
highest_freq = 1
num_chars_to_replace = window_length - highest_freq
                    = 1 - 1
                    = 0 ≤ k → expand
```



```
highest_freq = 2
num_chars_to_replace = window_length - highest_freq
                    = 2 - 2
                    = 0 ≤ k → expand
```



```
highest_freq = 2
num_chars_to_replace = window_length - highest_freq
                    = 3 - 2
                    = 1 ≤ k → expand
```



```
highest_freq = 2
num_chars_to_replace = window_length - highest_freq
                    = 4 - 2
                    = 2 ≤ k → expand
```

Once the window expands to the fifth character ('d'), it will contain 3 characters that must be replaced to make the window uniform. Since we can only replace up to k = 2 characters, the window is invalid. So, we shrink the window:

↑ ↑ ↑
chars to replace

$= 3 > k \longrightarrow \text{shrink}$



Notice that after shrinking the window, the value of **highest_freq** is still 2, which is no longer correct. Recall that our current method for updating **highest_freq** only increases it when encountering a character with a higher frequency, meaning **highest_freq** can only remain the same or increase, but it can never decrease.

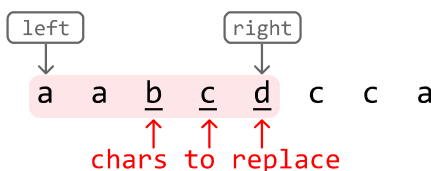
One way to work around this is to develop a new method for updating **highest_freq** that accurately decreases it when the highest frequency in a window decreases. However, our goal is to find the longest substring that meets the condition, so shrinking the window might not even be necessary. The crucial point here is that **when we find a valid window of a certain length, no shorter window will provide a longer uniform substring**.

This means we can just slide the window instead of shrinking it whenever we encounter an invalid window, effectively maintaining the length of the current window.

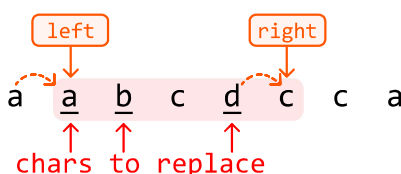
With this observation, we should correct our previous logic:

- If the window satisfies the condition: expand.
- **If the window doesn't satisfy the condition: slide.**

Let's correct the action taken in the first invalid window above by sliding instead of shrinking. Then, we can continue processing the rest of the string.



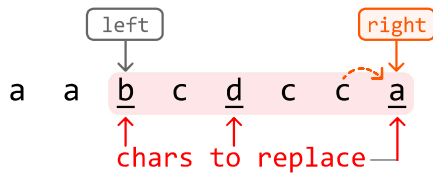
highest_freq = 2
 num_chars_to_replace = window_length - highest_freq
 = 5 - 2
 = 3 > k $\longrightarrow$ slide



highest_freq = 2
 num_chars_to_replace = window_length - highest_freq
 = 5 - 2
 = 3 > k $\longrightarrow$ slide

↑ chars to replace ↑

$= 2 \leq k \longrightarrow \text{expand}$



```
highest_freq = 3
num_chars_to_replace = window_length - highest_freq
                    = 6 - 3
                    = 3 < k → slide
```

The above window is the final window because we cannot expand or slide it any further. The longest valid window encountered during this process is a window of length 5.

Implementation

Python

JavaScript

Java

```
def longest_uniform_substring_after_replacements(s: str, k: int) → int:
    freqs = {}
    highest_freq = max_len = 0
    left = right = 0
    while right < len(s):
        # Update the frequency of the character at the right pointer and the high
        # frequency for the current window.
        freqs[s[right]] = freqs.get(s[right], 0) + 1
        highest_freq = max(highest_freq, freqs[s[right]])
        # Calculate replacements needed for the current window.
        num_chars_to_replace = (right - left + 1) - highest_freq
        # Slide the window if the number of replacements needed exceeds 'k'.
        # The right pointer always gets advanced, so we just need to advance 'left'
        if num_chars_to_replace > k:
            # Remove the character at the left pointer from the hash map before
            # advancing the left pointer.
            freqs[s[left]] -= 1
            left += 1
        # Since the length of the current window increases or stays the same, assign
        # the length of the current window to 'max_len'.
        max_len = right - left + 1
        # Expand the window.
        right += 1
    return max_len
```

Complexity Analysis

two pointers.

Space complexity: The space complexity is $O(m)$, where m is the number of unique characters in the string stored in the hash map `freqs`.

✓ Completed

[Teach on ByteByteGo](#)

[Be an Affiliate](#)

[Become a Contributor](#)

hi@bytebytego.com

[Report a Bug](#)

[Our Team](#)

[Newsletter](#)

[Privacy Policy](#)

[Terms of Service](#)

[Do Not Sell or Share My Personal Information](#)

[YouTube](#)

[Visual Dev Guides](#)